TP Guidé Spring Boot : StudyPlanner

Table of Contents

1. 1. Vision du Produit . . . 2
   1.1. Contexte du TP . . . 2
   1.2. Concepts Couverts . . . 2
2. 2. User Stories . . . 3
   2.1. US #1 – Planifier une Session . . . 3
   2.2. US #2 – Consulter mes Sessions . . . 3
   2.3. US #3 – Modifier une Session . . . 3
   2.4. US #4 – Supprimer une Session . . . 3
   2.5. US #5 – Validation des Données . . . 3
   2.6. US #6 – Sécurité des Données . . . 3
   2.7. US #7 – Monitoring de la Charge . . . 3
3. 3. Modèle de Données . . . 3
   3.1. 3.1 Entité StudySession . . . 3
   3.2. 3.2 Justification du Modèle . . . 4
4. 4. Spécifications API REST . . . 4
   4.1. 4.1 Endpoints StudySessions . . . 4
      4.1.1. POST /api/sessions . . . 4
      4.1.2. GET /api/sessions . . . 5
      4.1.3. GET /api/sessions/{id} . . . 6
      4.1.4. PUT /api/sessions/{id} . . . 6
      4.1.5. DELETE /api/sessions/{id} . . . 6
   4.2. 4.2 Format des Erreurs . . . 6
5. 5. Guide d'Implémentation . . . 7
   5.1. Étape 1 : Initialisation du Projet . . . 7
      5.1.1. 1.1 Créer le Projet . . . 7
      5.1.2. 1.2 Configuration de Base . . . 7
      5.1.3. 1.3 Structure des Packages . . . 7
   5.2. Étape 2 : Créer l'Entité StudySession . . . 8
   5.3. Étape 3 : Créer le Repository . . . 8
   5.4. Étape 4 : Créer le Service . . . 8
   5.5. Étape 5 : Gérer les Exceptions . . . 9
      5.5.1. 5.1 Créer les Exceptions Personnalisées . . . 9
      5.5.2. 5.2 GlobalExceptionHandler . . . 9
   5.6. Étape 6 : Créer le Controller . . . 10
   5.7. Étape 7 : Configurer la Sécurité . . . 10
   5.8. Étape 8 : Health Indicator Personnalisé . . . 11

5.9. Étape 9 : Tests . . . 11
5.10. 9.1 Tests Unitaires du Service . . . 11
5.11. 9.2 Tests Web du Controller . . . 12
6. 6. Livrables et Évaluation . . . 12
6.1. 6.1 Ce que Vous Devez Rendre . . . 12
6.2. 6.2 Grille d'Évaluation . . . 13
6.2.1. Palier 1 : Fondations (10/20) . . . 13
6.2.2. Palier 2 : API Complète (15/20) . . . 13
6.2.3. Palier 3 : Production Ready (20/20) . . . 14
6.3. 6.3 Bonus Appréciés (points supplémentaires) . . . 15
6.4. 6.4 Critères de Qualité Transversaux . . . 15
7. 7. Ressources . . . 16
7.1. Documentation Officielle . . . 16
7.2. Guides Utiles . . . 16
8. Conclusion . . . 17

1. 1. Vision du Produit

StudyPlanner est une application web permettant aux étudiants de planifier et organiser leurs sessions de révision. L'application offre une interface simple pour créer des créneaux de révision, les consulter et s'assurer qu'aucune session ne se chevauche.

1.1. Contexte du TP

- **Public** : Étudiants débutants ou intermédiaires en Spring Boot
- **Format** : Travail individuel ou binôme
- **Objectif** : Maîtriser les fondamentaux de Spring Boot
- **Approche** : Guidé pas-à-pas avec jalons progressifs

1.2. Concepts Couverts

- Architecture 3-tiers (Controller/Service/Repository)
- Spring Data JPA avec H2
- Bean Validation
- Exception Handling global
- Spring Security (Basic Auth)
- Spring Actuator
- Tests unitaires et d'intégration

2. 2. User Stories

2.1. US #1 – Planifier une Session

En tant qu'étudiant, je veux pouvoir créer une session de révision en indiquant la matière, la date et l'heure de début et de fin, afin d'organiser mon emploi du temps.

2.2. US #2 – Consulter mes Sessions

En tant qu'étudiant, je veux pouvoir consulter toutes mes sessions de révision planifiées afin de visualiser mon planning.

2.3. US #3 – Modifier une Session

En tant qu'étudiant, je veux pouvoir modifier les détails d'une session existante pour adapter mon planning.

2.4. US #4 – Supprimer une Session

En tant qu'étudiant, je veux pouvoir supprimer une session si mes plans changent.

2.5. US #5 – Validation des Données

En tant qu'étudiant, je veux être alerté immédiatement si je saisis des informations invalides (ex: date dans le passé, heure de fin avant heure de début).

2.6. US #6 – Sécurité des Données

En tant qu'étudiant, je veux que seules mes sessions soient visibles et modifiables par moi, pour garantir la confidentialité.

2.7. US #7 – Monitoring de la Charge

En tant qu'administrateur, je veux pouvoir vérifier la santé de l'application et détecter si un étudiant a une surcharge de sessions planifiées.

3. 3. Modèle de Données

3.1. 3.1 Entité StudySession

Champ	Type	Contraintes
id	Long	Clé primaire, auto-incrémenté

Champ	Type	Contraintes
studentName	String	Obligatoire, représente l'utilisateur connecté
subject	String	Obligatoire, nom de la matière (ex: "Java", "Mathématiques")
description	String	Optionnel, détails de la session
startTime	LocalDateTime	Obligatoire, doit être dans le futur
endTime	LocalDateTime	Obligatoire, doit être après startTime
createdAt	LocalDateTime	Auto-généré à la création
updatedAt	LocalDateTime	Auto-mis à jour à chaque modification

3.2. 3.2 Justification du Modèle

Pourquoi pas de relation User → StudySession ?

Pour simplifier le TP, nous n'utilisons pas d'entité `User` séparée. Le champ `studentName` contient le nom de l'utilisateur authentifié (récupéré via Spring Security).

Avantages : * Modèle simple avec une seule entité * Pas de gestion des relations JPA * Focus sur les autres concepts Spring Boot

En production : Une vraie application utiliserait une relation `@ManyToOne` vers une entité `User`.

4. 4. Spécifications API REST

4.1. 4.1 Endpoints StudySessions

4.1.1. POST /api/sessions

Créer une nouvelle session de révision.

Requête :

```
{
  "subject": "Java Spring Boot",
  "description": "Révision des concepts IoC et DI",
  "startTime": "2024-03-20T14:00:00",
  "endTime": "2024-03-20T16:00:00"
}
```

Réponse : 201 Created

```
{
  "id": 1,
```

```
  "studentName": "alice",
  "subject": "Java Spring Boot",
  "description": "Révision des concepts IoC et DI",
  "startTime": "2024-03-20T14:00:00",
  "endTime": "2024-03-20T16:00:00",
  "createdAt": "2024-03-15T10:30:00",
  "updatedAt": "2024-03-15T10:30:00"
}
```

Réponse : 400 Bad Request (validation échouée)

```
{
  "code": "VALIDATION_ERROR",
  "message": "Erreur de validation des données",
  "errors": {
    "subject": "Le sujet est obligatoire",
    "startTime": "La date de début doit être dans le futur"
  },
  "timestamp": "2024-03-15T10:30:00"
}
```

4.1.2. GET /api/sessions

Lister toutes les sessions de l'utilisateur connecté.

Réponse : 200 OK

```
[
  {
    "id": 1,
    "studentName": "alice",
    "subject": "Java Spring Boot",
    "description": "Révision des concepts IoC et DI",
    "startTime": "2024-03-20T14:00:00",
    "endTime": "2024-03-20T16:00:00",
    "createdAt": "2024-03-15T10:30:00",
    "updatedAt": "2024-03-15T10:30:00"
  },
  {
    "id": 2,
    "studentName": "alice",
    "subject": "Mathématiques",
    "description": "Algèbre linéaire",
    "startTime": "2024-03-21T09:00:00",
    "endTime": "2024-03-21T11:00:00",
    "createdAt": "2024-03-15T11:00:00",
    "updatedAt": "2024-03-15T11:00:00"
  }
```

```
]
```

4.1.3. GET /api/sessions/{id}

Obtenir les détails d'une session spécifique.

Réponse : 200 OK (session de l'utilisateur connecté)

Réponse : 404 Not Found (session inexistante ou appartient à un autre utilisateur)

4.1.4. PUT /api/sessions/{id}

Modifier une session existante.

Requête : Mêmes champs que POST

Réponse : 200 OK (session mise à jour)

Réponse : 404 Not Found (session inexistante ou appartient à un autre utilisateur)

4.1.5. DELETE /api/sessions/{id}

Supprimer une session.

Réponse : 204 No Content

Réponse : 404 Not Found (session inexistante ou appartient à un autre utilisateur)

4.2. 4.2 Format des Erreurs

Toutes les réponses d'erreur doivent suivre ce format :

```
{
  "code": "CODE_ERREUR",
  "message": "Message explicite",
  "errors": {
    "champ1": "Erreur spécifique",
    "champ2": "Autre erreur"
  },
  "timestamp": "2024-03-15T10:30:00"
}
```

Codes attendus : * `VALIDATION_ERROR` (400) : Données invalides * `NOT_FOUND` (404) : Ressource inexistante * `BUSINESS_ERROR` (422) : Erreur métier (ex: endTime avant startTime) * `UNAUTHORIZED` (401) : Non authentifié * `FORBIDDEN` (403) : Accès refusé * `INTERNAL_ERROR` (500) : Erreur serveur

5. 5. Guide d'Implémentation

5.1. Étape 1 : Initialisation du Projet

5.1.1. 1.1 Créer le Projet

Aller sur https://start.spring.io et configurer :

- **Project** : Maven
- **Language** : Java
- **Spring Boot** : 3.2.x ou supérieur
- **Java** : 17 ou supérieur
- **Group** : com.studyplanner
- **Artifact** : studyplanner

Dépendances à ajouter :

- Spring Web
- Spring Data JPA
- H2 Database
- Validation
- Spring Security
- Spring Boot Actuator

5.1.2. 1.2 Configuration de Base

Créer `src/main/resources/application.properties` avec la configuration H2, JPA, Actuator.

Ce que vous devez configurer :

- URL de connexion H2 en mémoire
- Activation de la console H2 pour débugger
- Configuration JPA (ddl-auto, show-sql)
- Exposition des endpoints Actuator (health, info)
- Détails du health visibles

5.1.3. 1.3 Structure des Packages

Organiser votre code selon l'architecture 3-tiers :

```
src/main/java/com/studyplanner/
 ├─── entity/
```

```
├─── repository/
├─── service/
├─── controller/
├─── exception/
├─── config/
└─── actuator/
```

5.2. Étape 2 : Créer l'Entité StudySession

Créer la classe `StudySession` dans le package `entity`.

Annotations JPA à utiliser :

- `@Entity` et `@Table` pour mapper l'entité
- `@Id` avec `@GeneratedValue(strategy = GenerationType.IDENTITY)` pour la clé primaire
- `@Column` pour configurer les contraintes (nullable, etc.)

Annotations de validation à ajouter :

- `@NotBlank` sur les champs obligatoires (studentName, subject)
- `@Future` sur startTime (empêche de planifier dans le passé)
- `@NotNull` sur les champs de date

Gestion automatique des timestamps :

Utiliser `@PrePersist` et `@PreUpdate` pour gérer automatiquement `createdAt` et `updatedAt`.

Note importante : La validation "endTime après startTime" doit être faite dans le service, pas au niveau de l'entité.

5.3. Étape 3 : Créer le Repository

Créer l'interface `StudySessionRepository` dans le package `repository`.

Ce que doit faire votre repository :

- Étendre `JpaRepository<StudySession, Long>`
- Ajouter une méthode pour récupérer toutes les sessions d'un étudiant
- Ajouter une méthode pour compter le nombre de sessions d'un étudiant

Astuce : Spring Data JPA génère automatiquement l'implémentation si vous respectez la convention de nommage (`findByNomDuChamp`).

5.4. Étape 4 : Créer le Service

Créer la classe `StudySessionService` dans le package `service`.

Principes à respecter :

- Annoter avec `@Service`
- Injecter le repository par constructeur
- Toute la logique métier doit être dans le service, pas dans le controller

Logique métier à implémenter :

Création d'une session :

- Affecter le nom de l'utilisateur connecté à la session
- Valider que `endTime` est après `startTime`
- Si la validation échoue, lancer une `BusinessException`
- Sauvegarder et retourner la session

Récupération des sessions :

- Pour la liste : utiliser la méthode du repository qui filtre par nom d'étudiant
- Pour une session spécifique : vérifier qu'elle existe ET qu'elle appartient à l'utilisateur connecté
- Si l'utilisateur essaie d'accéder à une session qui ne lui appartient pas, lancer `ResourceNotFoundException` (pour ne pas révéler son existence)

Modification et suppression :

- Toujours vérifier que la session appartient à l'utilisateur connecté
- Pour la modification, valider également les dates

5.5. Étape 5 : Gérer les Exceptions

5.5.1. 5.1 Créer les Exceptions Personnalisées

Créer deux classes d'exceptions dans le package `exception` :

- `ResourceNotFoundException` : pour les ressources non trouvées
- `BusinessException` : pour les erreurs métier (dates invalides, etc.)

Les deux doivent étendre `RuntimeException`.

5.5.2. 5.2 GlobalExceptionHandler

Créer une classe annotée `@RestControllerAdvice` pour intercepter toutes les exceptions.

Méthodes à créer :

Chaque méthode doit être annotée avec `@ExceptionHandler` pour un type d'exception spécifique et `@ResponseStatus` pour définir le code HTTP.

Exceptions à gérer :

- `MethodArgumentNotValidException` → 400 Bad Request
  - Parcourir les erreurs de validation
  - Extraire le champ et le message d'erreur
  - Retourner un objet ErrorResponse structuré
- `ResourceNotFoundException` → 404 Not Found
- `BusinessException` → 422 Unprocessable Entity
- `Exception` (générique) → 500 Internal Server Error

DTO ErrorResponse :

Créer un record avec : code, message, erreurs (Map), timestamp.

5.6. Étape 6 : Créer le Controller

Créer `StudySessionController` dans le package `controller`.

Configuration :

- `@RestController`
- `@RequestMapping("/api/sessions")`
- Injecter le service par constructeur

Point clé : Récupérer l'utilisateur connecté

Spring Security injecte automatiquement un objet `Principal` dans vos méthodes. Utilisez `principal.getName()` pour obtenir le nom de l'utilisateur authentifié.

Endpoints à créer :

- POST : Accepter `@Valid @RequestBody` + `Principal`, déléguer au service, retourner 201
- GET (liste) : Accepter `Principal`, déléguer au service, retourner 200
- GET (par ID) : Accepter `@PathVariable` + `Principal`, déléguer au service, retourner 200
- PUT : Accepter `@PathVariable` + `@Valid @RequestBody` + `Principal`, déléguer au service, retourner 200
- DELETE : Accepter `@PathVariable` + `Principal`, déléguer au service, retourner 204

Important : Le controller ne fait QUE de la validation syntaxique et de la délégation. Toute la logique métier est dans le service.

5.7. Étape 7 : Configurer la Sécurité

Créer `SecurityConfig` dans le package `config`.

Objectifs :

- Activer HTTP Basic Authentication
- Protéger les endpoints `/api/**` (authentification requise)
- Autoriser l'accès libre à `/h2-console/ et /actuator/`
- Créer des utilisateurs en mémoire pour les tests

Ce que vous devez implémenter :

Un bean `SecurityFilterChain` qui configure : * La désactivation du CSRF (pour simplifier les tests) * Les règles d'autorisation (requestMatchers) * L'activation de httpBasic

Un bean `UserDetailsService` qui retourne un `InMemoryUserDetailsManager` avec au moins 3 utilisateurs (alice, bob, charlie) ayant le rôle USER.

Un bean `PasswordEncoder` qui utilise `BCryptPasswordEncoder`.

Astuce : Pour permettre l'affichage de la console H2 dans un iframe, configurer les headers pour autoriser les frames de la même origine.

5.8. Étape 8 : Health Indicator Personnalisé

Créer `StudyLoadHealthIndicator` dans le package `actuator`.

Objectif : Surveiller le nombre total de sessions planifiées dans le système.

Implémentation :

- Annoter avec `@Component`
- Implémenter l'interface `HealthIndicator`
- Injecter le repository
- Dans la méthode `health()` :
  - Compter le nombre total de sessions
  - Retourner `Health.down()` si > 50 (surcharge critique)
  - Retourner `Health.up()` avec warning si > 30
  - Retourner `Health.up()` sinon
  - Ajouter des détails (totalSessions, status)

5.9. Étape 9 : Tests

5.10. 9.1 Tests Unitaires du Service

Créer `StudySessionServiceTest` dans `src/test/java`.

Configuration :

- `@ExtendWith(MockitoExtension.class)`
- `@Mock` le repository
- `@InjectMocks` le service

Scénarios à tester :

- Création réussie d'une session
- Rejet d'une session avec endTime avant startTime
- Récupération d'une session inexistante (doit lancer exception)
- Tentative d'accès à une session d'un autre utilisateur

5.11. 9.2 Tests Web du Controller

Créer `StudySessionControllerTest` dans `src/test/java`.

Configuration :

- `@WebMvcTest(StudySessionController.class)`
- `@MockBean` le service
- `@Autowired` MockMvc

Scénarios à tester :

- POST avec données valides + authentification → 201
- POST avec données invalides → 400
- GET sans authentification → 401

Astuce : Utiliser `@WithMockUser(username = "alice")` pour simuler l'authentification.

6. 6. Livrables et Évaluation

6.1. 6.1 Ce que Vous Devez Rendre

Code Source :

- Projet Maven complet et fonctionnel
- Tous les packages et classes mentionnés dans le guide
- Configuration `application.properties`

Tests :

- Minimum 5 tests unitaires (Service)
- Minimum 3 tests web (Controller)
- Tests bonus appréciés

Documentation :

- README.md avec :
  - Instructions pour lancer l'application
  - Exemples de requêtes curl avec réponses attendues
  - Utilisateurs de test (username/password)
  - Accès à H2 Console et Actuator

6.2. 6.2 Grille d'Évaluation

6.2.1. Palier 1 : Fondations (10/20)

Critères obligatoires :

- ☐ Projet Spring Boot démarre sans erreur
- ☐ Entité `StudySession` correctement définie avec tous les champs
- ☐ Annotations JPA appropriées (@Entity, @Table, @Id, @Column)
- ☐ Annotations de validation (@NotBlank, @NotNull, @Future)
- ☐ Gestion automatique des timestamps (@PrePersist, @PreUpdate)
- ☐ Repository avec méthodes personnalisées (findByStudentName, countByStudentName)
- ☐ Base H2 configurée et accessible via console
- ☐ Application.properties correctement configuré
- ☐ Au moins 2 tests unitaires

Points de vérification :

- L'entité respecte le modèle de données spécifié
- Les contraintes de validation sont testables
- La console H2 est accessible sur /h2-console
- Les tables sont créées automatiquement au démarrage

Livrables Palier 1 :

- Code source complet
- README avec instructions de démarrage
- Capture d'écran de la console H2 (optionnel)

6.2.2. Palier 2 : API Complète (15/20)

En plus du Palier 1 :

- ☐ Service avec toute la logique métier

- ☐ Validation métier : endTime après startTime
- ☐ Vérification que les sessions appartiennent à l'utilisateur connecté
- ☐ Exceptions personnalisées (ResourceNotFoundException, BusinessException)
- ☐ GlobalExceptionHandler fonctionnel avec @RestControllerAdvice
- ☐ Format d'erreur structuré (ErrorResponse avec code, message, timestamp)
- ☐ Controller avec tous les endpoints CRUD (5 endpoints)
- ☐ Récupération de l'utilisateur via Principal
- ☐ Sécurité Basic Auth implémentée
- ☐ Utilisateurs en mémoire créés (alice, bob, charlie)
- ☐ Au moins 5 tests unitaires + 3 tests web (MockMvc)

Points de vérification :

- Les erreurs de validation retournent 400 avec détails
- Les erreurs métier retournent 422
- Les ressources non trouvées retournent 404
- Alice ne peut pas voir les sessions de Bob
- Tests MockMvc utilisent @WithMockUser
- Basic Auth fonctionne (401 sans credentials)

Livrables Palier 2 :

- Exemples curl dans README pour tous les endpoints
- Démonstration de l'isolation des données par utilisateur
- Au moins 8 tests

6.2.3. Palier 3 : Production Ready (20/20)

En plus du Palier 2 :

- ☐ Health Indicator personnalisé fonctionnel
- ☐ Logique de surveillance implémentée (> 50 sessions = DOWN)
- ☐ Actuator configuré et accessible
- ☐ Endpoints health et info exposés
- ☐ Détails du health visibles
- ☐ Au moins 10 tests (unitaires + web)
- ☐ Tests couvrent tous les scénarios (succès + erreurs)
- ☐ Code propre et bien organisé
- ☐ Pas de code dupliqué

- ☐ Injection par constructeur respectée
- ☐ Documentation complète dans README

Points de vérification :

- L'endpoint /actuator/health affiche le health indicator custom
- Le statut change selon le nombre de sessions
- Les tests vérifient les cas limites (dates invalides, utilisateurs différents)
- Le code respecte les principes SOLID
- Le README permet à quelqu'un de démarrer l'application sans aide

Livrables Palier 3 :

- Application complète et robuste
- Tests exhaustifs avec bonne couverture
- Documentation professionnelle
- Code de qualité production

6.3. 6.3 Bonus Appréciés (points supplémentaires)

- Tests d'intégration avec @SpringBootTest et H2
- Logging structuré avec niveaux appropriés
- Pagination sur GET /api/sessions
- Filtre par date (sessions à venir uniquement)
- Détection des chevauchements de sessions
- Script shell pour tester tous les endpoints
- Validation custom avec annotation personnalisée
- Gestion des fuseaux horaires (ZonedDateTime)

6.4. 6.4 Critères de Qualité Transversaux

Code :

- Nommage cohérent et explicite (français ou anglais)
- Méthodes courtes (max 20 lignes)
- Une responsabilité par classe/méthode
- Pas de code dupliqué
- Injection de dépendances par constructeur uniquement
- Commentaires uniquement où nécessaire

Architecture :

- Séparation stricte des couches (Controller/Service/Repository)
- Pas de logique métier dans le Controller
- Pas d'accès direct au Repository depuis le Controller
- Exceptions métier gérées dans le Service

Tests :

- Tests unitaires isolés (mocks)
- Tests web avec MockMvc
- Nomenclature claire (shouldDoSomethingWhenCondition)
- Arrange/Act/Assert respecté
- Assertions pertinentes

Git :

- Commits atomiques avec messages clairs
- Pas de fichiers inutiles (.DS_Store, *.class, target/)
- `.gitignore` correctement configuré

Documentation :

- README avec sections claires
- Instructions de setup complètes
- Exemples d'utilisation
- Résolution des problèmes courants

7. 7. Ressources

7.1. Documentation Officielle

- Spring Boot : https://docs.spring.io/spring-boot/docs/current/reference/html/
- Spring Data JPA : https://docs.spring.io/spring-data/jpa/docs/current/reference/html/
- Spring Security : https://docs.spring.io/spring-security/reference/
- Bean Validation : https://jakarta.ee/specifications/bean-validation/3.0/

7.2. Guides Utiles

- Guide Spring Security : https://spring.io/guides/gs/securing-web/
- Guide REST API : https://spring.io/guides/gs/rest-service/
- Guide Actuator : https://spring.io/guides/gs/actuator-service/

8. Conclusion

Ce TP vous permet de maîtriser les fondamentaux de Spring Boot en construisant une application complète de bout en bout. Suivez le guide étape par étape, testez régulièrement, et n'hésitez pas à consulter la documentation officielle.

L'objectif est de comprendre comment les différentes couches (Entity, Repository, Service, Controller) interagissent, comment gérer les erreurs proprement, et comment sécuriser votre API.

Bon courage !